CHAPTER ONE

INTRODUCTION

Python is a **high-level scripting language**, (also called an Interpreter), which was developed by a renowned programmer, called **Guido van Rossum** Python in the early 1990's; and grew out of a project to design of a computer language that is **used** for a wide variety of **cybersecurity**, **data science** and **machine learning**, **artificial intelligence**, **game development, Scientific computing,** and **web development**, to mention but a few.

Unlike many similar languages, **Python's key language is very small and easy to master**, while allowing the addition of modules to perform a virtually limitless variety of tasks. It is a true object-oriented language (OOP) and is **available on a wide variety of platforms**. Python was meant to be easy for beginners to learn, yet powerful enough for even advanced users.

What is the difference between scripting and programming language?

Principally, all scripting languages are programming languages but not all programming languages are scripting languages. The basic difference between the two is that **scripting languages do not require the compilation step but are rather interpreted**. Instances of scripting languages conventionally used without an overt compilation step are **JavaScript**, PHP, VBScript and Python itself.

Python Releases

Python was first released on June 22 as Python 2.0.1, and was in use till today, December 8, 2022. The present version in use today is Python 3.11.1.

Main features of Python

Python's features include: -

a) **Easy-to-learn** − Python has few keywords, simple structure, and a clearly defined syntax. This allows the student to pick up the language quickly.
b) **Easy-to-read** − Python code is more clearly defined and visible to the eyes.
c) **Easy-to-maintain** − Python's source code is fairly easy-to-maintain.
d) **Possession of broad standard library** − Python's bulk of the library is very portable and cross-platform compatible on UNIX, Windows, and Macintosh.
e) **Interactivity** − Python has support for an interactive mode which allows interactive testing and debugging of snippets of code.
f) **Portability** − Python can run on a wide variety of hardware platforms and has the same interface on all platforms.

g) **Databases support** − Python provides interfaces to all major commercial databases.

h) **GUI Programming** − Python supports GUI applications that can be created and ported to many system calls, libraries and windows systems, such as Windows MFC, Macintosh, and the Window system of Unix.

i) **Scalability** − Python provides a better structure and support for large programs.

Python development environment

A Python development environment (IDE) typically consists of the tools and settings needed to write, test, and debug Python code. Here are the key components:

1. Python Interpreter: This is the core of the Python environment. It interprets and executes Python code. You need to have Python installed on your system to run Python scripts.
2. Text Editor or Integrated Development Environment (IDE): You write your Python code in a text editor or an IDE. Some popular editors include VSCode, Atom, Sublime Text, while popular IDEs include PyCharm, Jupyter Notebooks, and Spyder.
3. Version Control System (VCS): Git is commonly used for version control. It helps you track changes in your code, collaborate with others, and manage different versions of your project.
4. Package Manager: Python uses package managers like pip to install and manage third-party libraries or modules. This allows you to easily integrate external code into your projects.
5. Virtual Environment: This is an isolated environments (e.g., virtualenv or venv) which helps manage dependencies by creating isolated environments for different projects. This ensures that each project can have its own set of dependencies without interfering with others.
6. Debugger: Debugging tools help identify and fix errors in your code. Many IDEs come with built-in debuggers, and Python has a built-in debugger module (`pdb`).
7. Documentation Tools: Tools like Sphinx help generate documentation for your Python projects, making it easier for others to understand and use your code.
8. Testing Frameworks: Python has testing frameworks like pytest and unittest. These help you write and run tests to ensure the correctness of your code.
9. Build Tools: Tools like setuptools and pipenv assist in packaging and distributing your Python projects.

10. Continuous Integration (CI) Tools: Services like Jenkins, Travis CI, or GitHub Actions automate the process of building, testing, and deploying your code.

Setting up a Python development environment involves configuring these components according to your preferences and project requirements. Depending on your needs, you may choose a lightweight text editor for simple scripts or a feature-rich IDE for larger projects.

The differences between Python and other programming languages

Python is a special programming language which differs from the other languages as it is not confined to using keywords (like "begin" and "end" as in Pascal) or special characters (like ";" as in C++) as delimiters. Python uses "\" to indicate continuation of the previous line. Interestingly, Python is strict on indentation, which when not obeyed will make your program erroneous.

Why a development environment (IDE) is necessary

Python code needs to be written, executed and tested to build applications. This is achieved by the use of IDE. The IDE consists of the following functionalities:

a) A text editor which provides a way to write the code.
b) The interpreter which allows commands to be executed.
c) Test unit for testing to see if the written code does what it is expected to do.

Basic Python Structures

The visible structures are:

a) Conditional commands (if/else/elif),
b) Looping commands (while and for)
c) Error handling commands (try/except)

Comments in Python

Some lines of instructions that do not need to be executed or lines used as additional information within codes are referred to as comments. Different symbols are used as comments in different programming languages. For instance, Ada (--), BASIC (REM), C/C++ and Java (// , /*…*/) , ColdFusion <!--- comment --- >, MATLAB (%....), PASCAL [(*…..*)], PHP, R (#). Besides, Python uses (#) for its inline comments. For example,

```
# The following line prints Hello World on the screen
print("Hello, World!")
```

Modules and Packages

Python module is a Python's file with **.py** extension (or executable program) which contains variety of functions and data types, which can be called using the "import" command in order to do some tasks. The file name is the name of the module.

Examples of Python modules include:

1. Numpy
2. Pandas
3. Matplotlib
4. Scipy
5. Beautiful soup
6. Tkinker
7. Pygame
8. wxPython
9. Language binding
10. AST
11. Python Imaging Library
12. DBM
13. Pip
14. PySeries
15. PyGTK
16. bzip2
17. Pyglet

A Python **package** is a directory containing Python files and a file with the name **__init__.py**. Every package can contain one or more modules and the file.

Examples of Python packages include:

1. Pillow
2. Cycler
3. Kiwisolver
4. Matplotlib
5. Numpy
6. Pandas
7. Pip
8. Pyparsing
9. python-dateutil
10. Pytz
11. Setuptools
12. Six

Running Python programs

If running a Python program means you are calling Python to action, there are three basic ways of running Python programs, which include the following:

a) Python prompt (>>>)

This is the **direct interaction** with the Python interpreter where a user keys in command statements and instantly obtains the results.

For example:
>>> print ("Hello there! My name is Python.")

Output:
Hello there! My name is Python.

Python on the prompt can also be used as a **Handy Calculator** during the direct interaction, to execute and display the value of that expression as soon as it is typed, unless the value of an expression is assigned to a variable.

For example:
```
>>> books_number = 50
>>> unit_price = 25.75
>>> total_sell = books_number * unit_price
1,287.5
>>> 25 * (4 + 90) - 10
180
```

Exercise One:
Find the results of the following computations:

i) 5 + 2
ii) 50 - 5*6
iii) (50 - 5*6) / 4
iv) 8 / 5
v) 17 / 3
vi) 17 // 3
vii) 17 % 3
viii) 5 * 3 + 2
ix) 5. ** 2
x) 2**3.75 - 1

b) Text editor

Text editors (e.g. Note++, Sublime) are used to compose the Python codes and the file created saved with an extension .py (e.g. greetings.py). The saved file can be run on the command line as:

C:\Users*Your Name*>python greetings.py

Setting up the Environment

i) Click on Windows Explorer
ii) Click on This PC and go to installation folder of Python 3.7. In that folder you should see all the Python files.
iii) Copy that address starting with C: and ending with 3.7 and close that window.
iv) On the Windows Explorer, click on Computer→Properties→Advanced System Settings.
v) Click on Environment Variables.
vi) Under System Variables search for the variable **Path**.
vii) Select Path by clicking on it. Click on Edit.
viii) Scroll all the way to the right of the field called Variable value using the right arrow.
ix) Add a semi-colon (;) to the end and paste the path (to the Python folder) that you previously copied. Click OK.

Writing Your First Python Program

i) Create a folder called MyPythonProgs on your C:\ drive. This serves as the storage location for all your Python programs.
ii) Go to Start and type NotePad in the Start Search box at the bottom and press Enter, NotePad program opens.
iii) In Notepad type in the following program exactly as written:

```
# This file is for 'welcome.py'
print("Welcome Programmer!")
```

iv) Go to File and click on Save as.
v) In the field 'save in' browse for the C: drive and then select the folder MyPythonProgs.
vi) In the field File name remove everything that is there and type in Welcome.py.
vii) In the field Save as type select All Files
viii) Click on Save. You have just created your first Python program.

Running Your First Program

i) Go to Start and in the Search Box type cmd.
ii) A dark window will appear. Type cd C:\ and hit the key Enter.
iii) If you type dir you will get a listing of all folders in your C: drive. You should see the folder MyPythonProgs that you created.
iv) Type cd PythonPrograms and hit Enter. It should take you to the PythonPrograms folder.
v) Type dir and you should see the file Welcome.py.
vi) To run the program, type **python Welcome.py** and hit Enter.
vii) You should see the line 'Welcome Programmer!'
viii) Congratulations, you have run your Python program.

c) IDE tools

PyCharm, Jupyter etc. are IDE tools used to run a Python program. PyCharm for example, should be downloaded and installed after installing Python 3.x. PyCharm has three areas where programs are run. These include:

a) Console window
b) Local window and
c) Run window

CHAPTER TWO

INTRODUCTION

This chapter deals with Python data types and how to verify the type of data stored by any variable. The data type can be built-in or user defined.

Data Types

Data type is the **type of values stored program variables** which is characterized by name and value. The data that can be stored in computer memory can be of many types, i.e., numeric, and alphanumeric. For example, a **student roll number is stored as a numeric value and his or her address is stored as alphanumeric characters**. Python has various standard data types that are used to define the operations possible on them and the storage method for each of them. These data types include:

a) Int (integer)
b) Float
c) Boolean
d) String
e) Complex

Int (Integer):

Int, or integer, is a positive or negative whole number, without decimals and of unlimited length.

Example:

a) 24656354687656
b) -20
c) 0b10 # B or b is for Binary
d) 0X20 # X is for heXadecimal

Exercise Two:

Evaluate the following by printing the variables

a) x = -20 + 0b10
b) y = 2345 + (-30)
c) z = 0x20 * 20

Float:

Float, meaning "floating point number" is a positive or negative number, containing one or more decimals. Float can also be scientific numbers or numbers with exponential values, "e", to indicate the power of 10.

Example:

i) 1.2e4
ii) .025e1

Exercise Three:
Evaluate the following by printing the variables.

a) x = 2.8e2
b) y = 2.e-1
c) z = .25e3

Boolean:
Objects of Boolean type may have one of two values, True or False

Example:

a) x = 5
   x > 10
   False
b) a is A
c) 4.5 is not 45
d) Friends = [“Adamu”, “Sani”, “Umar”]
   “Sani” in Friends

Exercise Four:
Evaluate the following at the prompt

a) 2.8e2 > 2.3e-1
b) Name = True
   print(not(Name))
c) 2.67e2 == 2.67e-2
d) True and False

Strings:
Strings in Python are contiguous set of characters represented in the quotation marks. Python allows for either pairs of **single** or **double** quotes.

Example:

1. 'Hello' is the same as "Hello".
2. Strings can be output to screen using the **print()** function. For example: print("Hello").

Exercise Five:
Evaluate the followings:

1. print("Federal Polytechnic, Mubi")
2. print(‘Federal Polytechnic, Mubi ')
3. " "

In Python (and almost all other common computer languages), a **tab** character can be specified by the escape sequence **\t**:

Example:

a) >>> print("A\td\ta\tm\tu")
   A d a m u

b)
```
>>>Text1 = "U\tm\ta\tr"
>>>Text2 = Text1.expandtabs(2)
>>>print(Text2)
U       m       a       r
```

Complex:

Complex number literal is a data type in Python which mimics the mathematical notation, as the **standard form**, of a complex number. Complex numbers are of the form:

Complex = real_number + imaginary_number

For short, it can be re-written as:

z = a + bj

Why j instead of i?

Traditionally complex numbers use 'i' notation to represent the imaginary unit, but Python programming language uses 'j' instead. You might feel unease with Python's convention if you have a mathematical background. However, there are a few reasons that can justify Python's controversial choice:

a) The 'i' notation is already adopted by engineers. To avoid **name collisions** with electric **current**, 'j' is adopted by Python.
b) In computing, the letter 'i' is often used for **indexing** variable in loops.
c) The letter i can be easily confused with l or 1 in **source code**.

Example:

1.
```
x = 3 - 2j
y = 2 + 3j
print(x + y)
```
(5+1j)

2.
```
x = 3 + 2j
y = 2 - 3j
print(x - y)
```
(1+5j)

Exercise Six:

Evaluate the followings, given that x = 9 + 6j and y = 21 - 17j in the example below:
a) x – y
b) x * y
c) x/y

Object type verification

Python being interactive, can instantly detect and verify object type stored in a variable by using the **type()** function:

Example:

1. >>> x = -764590346
   >>> print(type(x))
   <class 'int'>

Exercise Seven:

Evaluate the followings using the example above:

a) y = 2.8
b) z = True
c) w = 3 + 2j
d) name = 'Adamu'

CHAPTER THREE

INTRODUCTION

Concepts of Variables and Constants

A variable is a **reserved memory location in a computer** meant to store different values to be referenced and manipulated by computer programs. Variables are characterized by identification, location, type and value.

Variables Scope and Namespaces

Python, like other programming languages, has two **types** of **variables**, i.e., built-in and user defined. During Python interactive session, when a variable is a typed one, python tries to perceive exactly what type of variable is used, to **prevent variables created by the user from overwriting or interfering with built-in** variables; this is called **conceptualization of multiple namespaces**. In order to maintain Python's records, it enforces what is known as the **LGB rule**, L for local namespace, G for Global namespace and B for built-in namespace functions and variables. For instance when x = 2.0 is typed, it is automatically perceived to be a "Float".

Rules and Naming Convention for Variables and constants

Like other programming languages, Python has rules to be observed before creating a variable.

These rules include:

a) Constants and variables names should comprise letters in lowercase (a - z) or uppercase (**A to Z**) or digits (**0 to 9**) or an underscore (_).

For example:

i) first_name
ii) Code_Block_9
iii) CamelBack
iv) Caps001_lock

b) Create a name that makes sense. For example, *number* makes more sense than just letter *n*.

c) If you want to create a variable name having two words, use underscore (_) to separate them. For example:

i) my_name
ii) current_salary

d) In forming Constant names, use capital letters where possible. For example:

i) PI
ii) G
iii) MASS
iv) SPEED_OF_LIGHT
v) TEMP

e) Never use special symbols like: ! @, #, $, %, etc. in forming a variable or constant name.

f) Do not start a variable name with a digit, e.g. 419_group, 234rays

Assigning Values to Variables

Python variables do not need explicit declaration to reserve memory space. The declaration happens automatically when you assign a value to a variable.
The equal sign **(=)** is used to **assign values** to variables. The value to the right of (=) can be a **single value** or an **expression.** An expression is a combination of **values, variables, and operators** to be evaluated using assignment operator. For instance: y = x + 17, shows that y is the variable and x + 17 is the value; when x = 10, y will be evaluated to 30. So, a value in itself is a simple expression.
Example:

```
>>> x = 10
>>> y = x + 20
>>> y
30
```

The operand to the left of the (=) operator is the **name** of the variable and the operand to the right of the (=) operator is the **value** stored in the variable.

Exercise Eight

```
A = 400                 # An integer assignment
b = 20.0                # A floating point
c = "Adamu"     # A string
print (a)
print (b)
print (c)
```

Multiple Assignments to variables

In Python, single value can be assigned several variables simultaneously. For example: a = b = c = 1

Here, **three variable objects** a, b, and c are assigned same value 1, and **all** are assigned to the **same memory location**.

You can also assign multiple objects to multiple variables.

For example: a, b, c = 1, 2, "market“

Here, **two integer objects** with values **1 and 2** are assigned to variables **a** and **b** respectively, and **one string object** with the value "market" is assigned to the variable **c**.

Outputting Variables

The Python **print()** statement is often used to **output variables contents**. Variables do **not need to be declared** with any particular type and can even **change type** after they have been set.

```
x = 5                    # x is of type integer
x = "Polytechnic"        # x is now of type string
print(x)
```

Output: Polytechnic

Constants

A constant is a type of variable whose value does not change during program execution. It is useful to think of constants as containers that hold information which cannot be changed later.

Declaring and assigning values to constants

In Python, a constant is usually declared by typing its name in capital letter and join with underscore (in a compound name) and assigned values in a program.

Example:

```
PI = 3.14
GRAVITY = 9.8

print(PI)
print(GRAVITY)
```

Output

```
3.14
9.8
```

Naming the constants in **capital letters** is a **convention** to **separate them from variables**, however, it does not actually prevent reassignment.

Concept of Type Casting in Python

The technique of converting one data type into the other data type is known as Type Casting in python. Python supports a wide variety of functions or methods for such conversions. Such functions include: int(), float(), str(), ord(), hex(), and oct().

Python has **two basic types** of type conversion, namely, **Implicit** Conversion and **Explicit** Conversion.

Implicit Type casting

Implicit type conversion is a type of conversion performed **automatically** by the Python interpreter, **without the user intervention**. Python promotes the conversion of lower data type, for example, integer, to higher data type, say float to avoid data loss. This type of **conversion** is called **Up-Casting**.

Example:

```
# Python converts ‘a’ to ‘int’
a = 5
>>> print(a)
5
>>> print(type(a))
<class 'int'>

# Python converts ‘b’ to ‘float’
>>> b = 3.5
>>> print(b)
3.5
>>> print(type(b))
<class 'float'>

# Python converts addition of int and float to ‘float’
>>> c = a + b
>>> print(c)
8.5
>>> print(type(c))
<class 'float'>

# Python converts multiplication of ‘int’ and ‘float’ to ‘float’
>>>a = 5
>>>b = 3.5
>>> c = a * b
Print(c)
17.5
>>> print(type(c))
<class 'float'>
```

Exercise Nine

Evaluate the followings:

i) int_num = 95
   print(type(int_num))
ii) str_num = "756"
   print(type (str_num))
iii) flo_num = .57
   print(type (flo_num))
iv) bin_num = 0b100111
   print(type (bin_num))

v) oct_num = 55053
print(type(oct_num))
vi) cpl_num = 3+2j
print(type(cpl_num))

Explicit Type casting

In Explicit Type conversion, the user or programmer converts the data type of an object to the required data type. In Python, predefined functions like int(), float(), str(), etc. are used to perform explicit type conversion.

Example:
Consider the following string objects x and y, where:

```
>>> x = '100'
>>> y = '-90'
>>> print(x + y)
100-90
>>> print(int(x) + int(y))
10
```

From the above example:

1. x and y are initially strings, which was converted to integer using the int() function.
2. Same strings can be converted to float using the function, float() which does basically the same thing:

```
>>> print (float(x) + float(y))
10.0
```

Exercise Ten
Evaluate the followings:

i) int_num = 15
float(int_num)
ii) str_num = "56"
int(str_num)
iii) flo_num = 35.5
bin(flo_num)
iv) bin_num = 1100100111
hex(bin_num)
v) oct_num = 305354
float(number)

CHAPTER FOUR

INTRODUCTION

Data Structures are techniques of organizing data so that it can be easily and efficiently accessed. Fundamentally, all programming language rely on data structures about which programs are built.

Python Data Structures

Data structures are generally considered as "containers" that organize and group data according to type. Python **data structures** include: Lists, Dictionary, Tuple, and Set.

Python list

List is an ordered and changeable (**mutable**) collection which allows for **duplicate** membership. Python lists are structures that store **multiple elements** in a single variable. List variables can be **as long as needed** whose individual elements can be of **homogeneous or not**. The items in a list are separated by **commas** and enclosed in **square** brackets.

Example:

a) Books = ["Python", "C++", "Java", "Oracle"]
b) Numbers = [1, 2, 3, 4, 5]
c) Alphabets = [a, b, c, d, e]
d) List1 = ['a','b','c','d', [6, 7]]
e) List2 = [7, 'hi', None, False, True, ['Lion', 'Panda']]

Accessing items of a list

To access an element in a list, we use list_name, bracket notation and an index. For example:

i) General form:
   list_name[index]
ii) Specific form:
   Friends = ["Adamu", "Ahmed"]

Lists in Python use a **zero-based** index,

Example:

1. Mylist = ['a', 1, False]
   print(Mylist [0]) # Output: 'a'

2. Mylist = ['a', 1, False]
   print(Mylist [1]) # Output:1

3. Mylist = ['a', 1, False]
   print(Mylist [2]) # Output: False

List Methods

In addition, there are a number of inbuilt **methods** you can use to manipulate LIST. Methods are called using the **dot (.)** operator.

Examples:

1. **append**() - adds a value to the end of a list:

```
>>>MyList1 = [11,25,3]
>>>MyList1.append(10)
>>>print(Mylist1)
```

Output:
[11,25,3,10]

2. **clear**() - removes all the values in a list:

```
>>>MyList2= [1,2,3]
>>>MyList2.clear()
>>>print(MyList2)
```

Output:
[]

Exercise Eleven:

Evaluate the following **List** methods:

1. **copy**() - makes a shallow copy of a list:

```
MyList = [2,3,4]
YourList = MyList.copy()
print(YourList)
```

2. **count**() - counts how many times an item appears in a list:

```
MyList3 = [1,2,3,4,5,5,5,5,5]
MyList3.count(3)
print(MyList3.count(5))
```

3. **extend**() - adds all elements of a list to some other list:

```
MyList4 = [1,2,3]
MyList4.extend([5,6,7])
print(MyList4)
```

4. **index**() - returns the index of an element. If the element is not found, index returns a ValueError:

```
MyList5 = [4,8,6,4]
print(MyList5.index(8))
print(MyList5.index(4))
print(MyList5.index(12))
```

5. **insert**() - inserts an element to the list at the given index:

```
MyList6 = [4,6,8]
MyList6.insert(2,'awesome')
print(MyList6)
```

6. **pop**() - removes the last element from a list and returns the element removed:

```
MyList7 = [3,4]
print(MyList7.pop())
print(MyList7.pop())
```

Note: pop() throws an error if you call it on an empty list.

7. **remove**() - removes the first occurrence of a value:

```
MyList8 = [1,2,3,1]
MyList8.remove(1)
print(MyList8)
```

8. **reverse**() - reverses the order of all elements of the list:

```
MyList9 = [1,2,3,4]
MyList9.reverse()
print(MyList9)
```

9. **sort**() - sorts all elements of a list in the ascending order:

```
MyList10 = [1,4,3,2]
MyList10.sort()
print(MyList10)
```

Python Tuple

Tuple in Python, is also an **orderly** collection of items with the following features:

i) the tuples are **immutable**, which means we cannot change the elements once assigned and
ii) Tuples use **parentheses**, whereas lists use square brackets.

Because of immutability, operations on tuples are faster than on lists.

Example:

a) Books = (“Python”, “C++”, “Java”, “Oracle”, “Java”)
b) Numbers = (1, 2, 3, 4, 5)
c) Alphabets = (‘a’, ‘e’, ‘b’, ‘c’, ‘e’, ‘e’, ‘d’, ‘e’)
d) Tuple1 = ('a','b','c','d', (6, 7))
e) Tuple2 = (7, 'hi', None, False, True, ('Lion', 'Panda'))

To write a tuple containing a **single value** you have to **include a comma, even though there is only one value**:

Tuple2 = ('a',)

Unlike LIST the common methods used on tuples include:

1. **count**() - returns the number of times a value appears in a tuple:

```
Numbers = (1, 2, 3, 3, 3, 5, 5)
print(Numbers.count(1))          #1
print(Numbers.count(3))          #3
print(Numbers.count(5))          #2
```

2. **index**() - returns the index of the given element in the tuple:

```
Books = (“Python”, “C++”, “Java”, “Oracle”, “Java”)
print(Books.index(“C++”))        # 1
print(Books.index(“Oracle”))     #3
```

3. **len**() - returns the length of a tuple:

```
Books = (“Python”, “C++”, “Java”, “Oracle”, “Java”)
print(len(Books))                # 5
```

Exercise Twelve:

Evaluate the following **Tuple** methods:

i) Alphabets = (‘a’, ‘e’, ‘b’, ‘c’, ‘e’, ‘e’, ‘d’, ‘e’)
print(Alphabets.count(‘e’))
ii) Tuple1 = (2, 3, 5, 6, 7, 9)
print(len(Tuple1))
iii) Tuple2 = (7, 'hi', None, False, True)
print(Tuple2.index(False))

Python Dictionary

Dictionary in Python is a **key value pair** which is defined using **curly braces**, separating main value pairs with a **comma**, and placing a **colon** between the key and the value:

The following examples all return a dictionary equal to {"one": 1, "two": 2, "three": 3}:

```
>>> a = dict(one=1, two=2, three=3)
>>> b = {'one': 1, 'two': 2, 'three': 3}
>>> c = dict(zip(['one', 'two', 'three'], [1, 2, 3]))
>>> d = dict([('two', 2), ('one', 1), ('three', 3)])
>>> e = dict({'three': 3, 'one': 1, 'two': 2})
>>> f = dict({'one': 1, 'three': 3}, two=2)
```

Example:

```
i) >>> a = dict(one=1, two=2, three=3)
   >>>type(a)
```

Output

```
<class 'dict'>
```

Exercise Thirteen:

Verify the above dictionary formats ‘b’ to ‘d’ by using **type()** function as in the example above:

```
i)      b = {'one': 1, 'two': 2, 'three': 3}
ii)     c = dict(zip(['one', 'two', 'three'], [1, 2, 3]))
iii)    d = dict([('two', 2), ('one', 1), ('three', 3)])
iv)     e = dict({'three': 3, 'one': 1, 'two': 2})
v)      f = dict({'one': 1, 'three': 3}, two=2)
```

A typical instance of a dictionary:

```
year_months = {
   1: 'January',
   2: 'February',
   3: 'March',
   4: 'April',
   5: 'May',
   6: 'June',
```

```
    7: 'July'
    8: 'August',
    9: 'September',
    10: 'October',
    11: 'November',
    12: 'December',
}
```

It is worth noting from the above example that:

a) year_months: is the **Dictionary** name
b) 1, 2, etc. are the Keys followed by colon
c) 'January', 'February', etc. are the values associated with the Keys
d) keys in a dictionary must be unique (no two same keys)
e) keys are immutable
f) keys and values can be of any data type
g) the **keys() function** returns a list of keys in a dictionary
h) the **values()** function returns a list of values in a dictionary

Followings are the common **methods** on dictionaries:

1. **clear()** - clears all the keys and values in a dictionary:

```
old_depts = {
  'cs': ‘Computer Science’,
  'hmt': Hospitality Management',
  'slt': 'Science Lab Technology',
}
old_depts.clear()
print(old_depts)
```

Output:

```
#{}
```

2. **copy()** - makes a copy of a dictionary:

```
old_depts = {
  'cs': ‘Computer Science’,
  'hmt': Hospitality Management',
  'slt': 'Science Lab Technology',
}
new_depts  = old_depts.copy()
print(new_depts)
```

Exercise Fourteen:

Evaluate the following **Dictionary** methods:

1. **fromkeys()** - creates key value pairs from comma separated values:

```
keys = ('a', 'e', 'i', 'o', 'u')
value = 'vowel'
vowels = dict.fromkeys(keys, value)
```

2. **get()** - retrieves a key in an object and returns value:

```
old_depts = {
  'cs': ‘Computer Science’,
  'hmt': Hospitality Management',
  'slt': 'Science Lab Technology',
}
new_depts = old_depts.get('hmt')
print(new_depts)
```

3. **items()** - returns a list of tuples with each key-value pair:

```
old_depts = {
  'cs': ‘Computer Science’,
  'hmt': Hospitality Management',
  'slt': 'Science Lab Technology',
}
new_depts = old_depts.items()
print(new_depts)
```

4. **keys()** - returns a dict_keys object containing all of the keys in an object:

```
old_depts = {
  'cs': ‘Computer Science’,
  'hmt': Hospitality Management',
  'slt': 'Science Lab Technology',
}
new_depts = old_depts.keys()
print(new_depts)
```

5. **setdefault()** - returns the value of the specified key:

```
old_depts = {
  'cs': ‘Computer Science’,
  'hmt': Hospitality Management',
  'slt': 'Science Lab Technology',
}
new_depts = old_depts.setdefault(‘coe’, ‘Computer Engineering’)
print(new_depts)
```

6. **update()** - update keys and values in a dictionary with another set of key value:

```
old_depts = {
  'cs': ‘Computer Science’,
  'hmt': Hospitality Management',
  'slt': 'Science Lab Technology',
}
new_depts = old_depts.update(‘coe’: ‘Computer Engineering’)
print(new_depts)
```

7. **values()** - returns a list of all the values in the dictionary:

```
old_depts = {
  'cs': ‘Computer Science’,
  'hmt': Hospitality Management',
  'slt': 'Science Lab Technology',
}
new_depts = old_depts.values()
print(List(new_depts))
```

Python Set

Set can be defined as a collection of unique elements that do not follow a specific order. In addition to being iterable and mutable, **sets do not allow duplicate values**.

Set methods with examples

1. **add()** - adds an element to a set. If the element is already in the set, the set does not change:

```
set1 = {1,2,3}
set1.add(4)
print(set1)                          #{1, 2, 3, 4}
```

2. **clear()** - removes all the items of the set:

```
set2 = {1,2,3}
set2.clear()
print(set2)                          #set() i.e. empty set
```

3. **copy()** - creates a copy of the set:

```
set3 = {1,2,3}
set4 = set3.copy()
print(set4)                          #{1, 2, 3}
```

Exercise Fifteen:

Evaluate the following **SET** methods:

1. **difference()** - returns a new set with all the elements that are in the first set but not in the second set:

```
set1 = {1,2,3}
set2 = {2,3,4}
x = set1.difference(set2)
print(x)
y = set2.difference(set1)
print(y)
```

2. **intersection()** - returns a new set containing all the elements that are in both sets:

```
set1 = {1,2,3}
set2 = {2,3,4}
set3 = set1.intersection(set2)
print(set3)
```

3. **union()** - returns a union of two sets:

```
set1 = {1,2,3}
set2 = {2,3,4}
set3 = set1.union(set2)
print(set3)
```

Operators in Python

Python, like any other programming language, uses operators (standard symbols) to perform operations on values and variables. These include: arithmetic, comparison, logical, assignment, membership and identity operators.

Arithmetic operators

Arithmetic operators are used to perform mathematical operations like addition, subtraction, multiplication, etc. Operators are special symbols (e.g. +, -, *, /,) in Python that carry out arithmetic or logical computation. The value that the operator operates on is called the operand.

Example:

a)
```
>>> a = 5
>>> b = 7
>>> a + b
12
```

b)
```
>>> a = 9
>>> b = 7
>>> a - b
2
```

Exercise Sixteen:

Evaluate the following **Arithmetic Operators**:

1.
```
a = 15     # '*' multiplies left operand by right operand
b = 7
print(a * b)
```

2.
```
a = 25     # '/' divides left operand by the right one (results into float)
b = 17
print (a / b)
```

3.
```
a = 35     # '%' Modulo division - remainder after dividing left operand by
b = 27       right one
print (a % b)
```

4.
```
a = 45     # '//' Floor division: divides left by right and results into whole
b = 37       number adjusted to the left in the number line
print (a // b)
```

5.
```
a = 65     # '**' Exponent - left operand raised to the power of right
b = 47
print (a ** b)
```

Comparison operators

Comparison operators are used to compare values. It returns either True if the condition is met or False otherwise.

Example:

a)
```
a = 9
b = 7
a > b
```

Output:
True

b)
```
a = 9
b = 7
a < b
```

Output:
False

Exercise Seventeen:

Evaluate the following **Comparison Operators**:

1.
```
a = 15
b = 7
a == b
```

2.
```
a = 25
b = 17
a != b
```

3.
```
a = 35
b = 27
a >= b
```

4.
```
a = 45
b = 37
a <= b
```

Logical operators

Logical operators are the 'and', 'or', 'not' operators.

Example:

'and' operator returns True if both the operands are True

a)
```
a = True
b = True
print(a and b)
```

Output:
True

'or' operator returns True if either of the operands is True

b)
```
a = True
b = False
print(a or b)
```

Output:
False

'not' operator returns True if operand is false

c)
```
a = False
print(not a)
```

Output:
True

Exercise Eighteen:

Evaluate the following **Logical Operators**:

a)
```
a = False
b = True
print(a and b)
```

b)
```
a = False
b = False
print(a or b)
```

'not' operator returns True if operand is false

c)
```
a = True
print(not a)
```

Assignment operators

Python uses assignment operators to assign values to variables. Simply writing w = 5, Python assigns the value 5 on the right to the integer variable 'w' on the left. There are various compound operators associated with Python, for instance, w+= 5, means adding 5 to the variable and later assigns the same. It is equivalent to w = w + 5.

Example:

'a+=2' assignment implies **a = a + 2**

a)
```
b = 25
b += 25
print(b)
```

Output:
50

'a*=2' assignment implies **a = a * 2**

b)
```
b = 25
b *= 25
print(b)
```
Output:
625

'a%=2' assignment implies **a = a % 2**

c)
```
b = 25
b %= 25
print(b)
```
Output:
0

Exercise Nineteen:

Evaluate the following **Assignment Operators**:

a)
```
b = 35
b //= 35
print(b)
```

b)
```
b = 35
b **= 35
print(b)
```

c)
```
b = 35
b &= 35
print(b)
```

d)
```
b = 35
b |= 35
print(b)
```

e)
```
b = 35
b ^= 35
print(b)
```

f)
```
b = 35
b %= 35
print(b)
```

Identity operators

Python programming language is composed of some special type of operators, namely the **identity operators** symbolized as '**is**' and '**is not', which** are used to check if two values (or variables) are **located on the same part of the memory**. Two variables that are equal does not imply that they are identical.

Operator	Meaning	Example
is	True if the operands are identical (refer to the same object)	x is True
is not	True if the operands are not identical (do not refer to the same object)	x is not True

Example:

a)
```
x1 = 5
y1 = 5
print(x1 is y1)
```

Output:
True **# i.e. x1 and y1 are identical integers**

b)
```
x2 = 'Hello'
y2 = 'Hello'
```

```
print(x2 is y2)

Output:                    # i.e. x2 and y2 are identical strings
True
```

c)
```
x3 = [1,2,3]
y3 = [1,2,3]
print(x3 is y3)

Output:                    # i.e. x3 and y3 are equal lists but not identical
False                            because the interpreter locates them separately
                                 in memory
```

Exercise Twenty:

Evaluate the following **Identity Operators**:

a)
```
x1 = 55
y1 = 55
print(x1 is not y1)
```

b)
```
x2 = 'Welcome'
y2 = ' Welcome '
print(x2 is not y2)
```

c)
```
x3 = [10,20,30]
y3 = [10,20,30]
print(x3 is not y3)
```

Conditional Statements in Python

Conditional statements are also the decision-making statements. Conditional statements are used to execute the specific block of code depending on the given condition being true or false.

In Python, decision making statements include the following:

i) if statements
ii) if-else statements
iii) elif statements
iv) Nested if and if-else statements

IF statements without ELSE

Python IF statement decides whether certain statements need to be executed or not. It checks the CONDITION of a given BOOLEAN expression, if the condition is TRUE, then the set of instructions inside the IF block will be executed otherwise not.

Syntax:

If (EXPRESSION == TRUE):
    Block of code to be executed

Example:

```
Age = 5
if Age < 7:
    print("You are in Nursery school")
```

Output:
You are in Nursery school

Example:

The following code checks for a membership of an element in a list:

```
if 'Python' in ['Java', 'Python', 'C#']:
    print("True")
```

Output:
True

IF statements with ELSE

Here Python checks the CONDITION of a given BOOLEAN expression, if the condition is TRUE, then the set of instructions inside the IF block will be executed and if the CONDITION is FALSE, ELSE block then becomes executed.

Syntax:

```
If ( EXPRESSION = VALUE ):
        Block of code to be executed
else:
        Block of code to be executed
```

Example:

```
age = 15
if (age < 7):
        print("You are in Nursery school")
else:
        print("You are in Primary school")
```

Output:
You are in Primary school

Example:

```
age = 20
if (age >= 19):
        print("You can vote this year")
else:
        print("Pls wait for your turn")
```

Output:
You can vote this year

Example:

```
a = 90
b = 70
if (a < b):
        print("a is smaller than b" )
else:
        print("b is smaller than a" )
```

Output:
b is smaller than a

ELIF statements

When more than one conditions are to be evaluated, Python uses an ELIF statements. It is used to check multiple conditions only if the given condition is false. ELIF block always ends with ELSE.

Syntax:

if (condition):
   #Set of statement to be executed if condition is true
elif (condition):
   #Set of statements to be executed when if condition is false and elif condition is true
else:
   #Set of statement to be executed when both if and elif conditions are false

Example:

```
my_marks = 90
if my_marks < 40:
    print("Sorry!, You failed the exam")
elif not my_marks < 40 and my_marks < 90:
    print("You passed the exam! ")
else:
    print("Passed in First class with distinction")
```

Output:

Passed in First class with distinction

Multiple Conditions in IF Statements

Python allows for evaluation of multiple conditions in an IF statement like below.

Example:
Consider the candidate seeking for admission to do a master's programme in a university. A candidate can only be given admission if he/she has Bachelor's degree certificate is available, Transcript for undergraduate degree, First degree in the chosen course of study or PGDE in same course and has a CGPA >=3.0.

```
own_bach_degree = True
trans_avail = True
degree_chosen_course = True
PGDE_chosen_course = True
CGPA = 3.0

if own_bach_degree and trans_avail and degree_chosen_course and CGPA < 3.0:
    print("Admission Denied")

elif not degree_chosen_course and own_bach_degree and trans_avail  and CGPA >= 3.0:
    print("Admission Denied")

elif not (trans_avail and degree_chosen_course) and own_bach_degree  and CGPA >= 3.0:
    print("Admission Denied")

elif not degree_chosen_course and own_bach_degree and trans_avail  and CGPA < 3.0:
    print("Admission Denied")

elif own_bach_degree and trans_avail and degree_chosen_course and CGPA >= 3.0:
    print("Admission Recommended")

elif own_bach_degree and trans_avail and PGDE_chosen_course and CGPA >= 3.0:
    print("Admission Denied")
else:
    print("You did not apply for Admission")
```

Exercise Twenty-One:

Evaluate the following conditional statement

1.
```
Number = 11
If (Number > 8):
    print("The square of the numbers is:    ", Number * Number )
```

2.
```
if ('Ahmed' in ['Jalo', 'John', 'Ahmed', 'Kassim']):
    print("True")
```

3.
```
age = 18
own_card = True
if (age >= 18 and own_card == True):
    print("You can vote this year")
elif (age >= 18 and own_card == False):
    print("Please you are not eligible to vote ")
else:
    print("Pls wait for your turn!")
```

4.
```
curYear = int(input (" Enter the year: " ) )
month = int(input("Enter the month: " ) )
if ((curYear % 4 ) == 0 and (curYear % 100 ) != 0 or (curYear % 400 ) == 0 ):
        print("Leap Year")
    if(month == 1 or month == 3 or month == 5 or month == 7 or month == 8 or month
       == 10 or month == 12):
        print("There are 31 days in this month " )
    elif ( month == 4 or month == 6 or month == 9 or month == 11 ):
        print("There are 30 days in this month " )
    elif ( month == 2 ):
        print("There are 29 days in this month " )
    else:
        print("Invalid month ")
elif ( (curYear % 4 ) != 0 or (curYear % 100 ) != 0 or (curYear % 400 ) != 0 ):
    print("Non Leap Year " )
        if ( month == 1 or month == 3 or month == 5 or month == 7 or month == 8 or
           month == 10 or month == 12 ):
            print("There are 31 days in this month" )
        elif ( month == 4 or month == 6 or month == 9 or month == 11 ):
            print("There are 30 days in this month " )
        elif ( month == 2 ):
            print("There are 28 days in this month ")
        else:
            print("Invalid month " )
else:
 print( " Invalid Year " )
```

Loops in Python (For Loops, While Loops)

Python has two primitive loop commands:

a) while loops
b) for loops

The while Loop

With the while loop we can execute a set of statements as long as a condition is true.

Example:

Print even numbers less than 30:

```
i = 2
while i < 15:
    print(i)
i += 2
```

Output:

```
2
4
6
8
10
12
14
```

WHILE Loop with BREAK Statement

Break statement stops While loop even if the while condition is true:

Example:

The following code Exits While loop when ‘i' is 12:

```
i = 1
while i < 24:
  print(i)
  if i == 12:
    break                # values stop at 12 despite length 24
  i += 1
```

Output:

1
2
3
4
5
6
7
8
9
10
11
12

WHILE Loop with CONTINUE Statement

Current iteration can be stopped with the CONTINUE statement, and continue with the next:

Example

Continue to the next iteration if i is 12, skipping the tested equality value:

```
i = 0
while i < 24:
  i += 1
  if i == 12:      # Jumps 12 and continues the iteration
    continue
  print(i)
```

Output:

Predict the output

WHILE Loop with ELSE Statement

With ELSE statement a block of code can be ran once when a condition is FALSE:

Example:

```
i = 1
while i < 6:                    # stops at 5 iteration
  print(i)
  i += 1
else:
  print("i is no longer less than 6")
```

Output:

Predict the output